جامعة الوادي — كلية العلوم الدقيقة — قسم الإعلام الآلي

تقرير العمل التطبيقي (TP): نماذج التصميم Design Patterns

مشروع UniAccounts: نظام إدارة حسابات الطلبة — العمل الأول: النواة وأنماط الإنشاء

الطالب: الحفناوي دوقة **الفوج:** 1 **المستوى:** السنة الثالثة ليسانس (LMD) إعلام آلي **التاريخ:** 05/11/2026

1. تقديم المشروع والهدف

تريد الجامعة نظاماً صغيراً لإدارة حسابات الطلبة. عند التحاق طالب جديد يُنشأ له حساب يحمل معلوماته الشخصية والأكاديمية، وتُحفظ كل الحسابات في مستودع مركزي، وتُسجَّل كل العمليات في سجل تدقيق (Audit Log). الهدف من هذا العمل ليس كتابة برنامج كبير، بل **استعمال نمط التصميم في الموضع الذي يحل فيه مشكلة حقيقية** وفهم سبب اختياره. أنجزنا في هذا الجزء نواة النظام بثلاثة أنماط إنشائية (Creational).

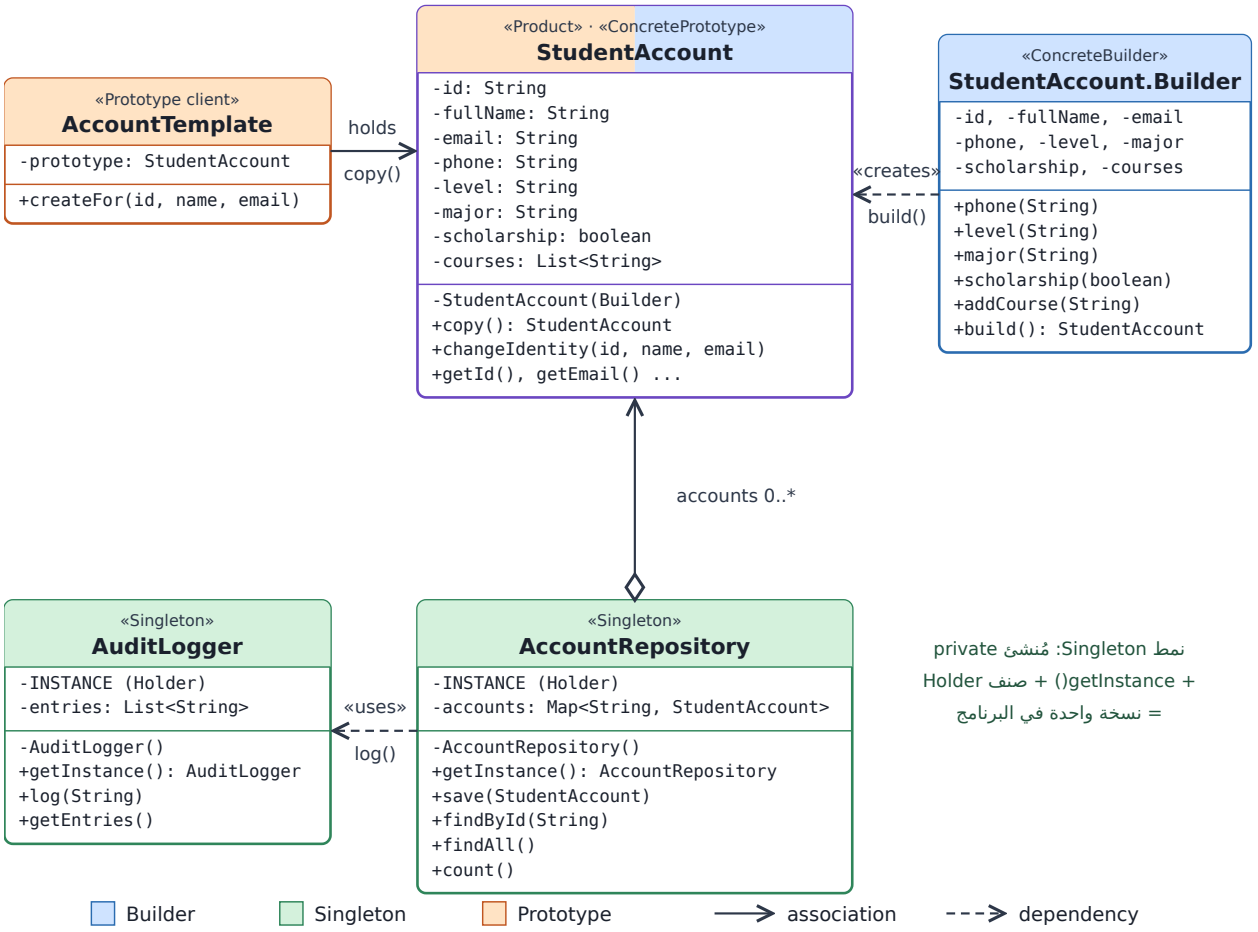
2. المشاكل المطروحة والأنماط المختارة

النمط	المشكلة في المشروع	الحل في الكود
Builder	حساب بثمانية حقول بعضها إجباري وبعضها اختياري؛ مُنشئ بثمانية وسائط غير مقروء وعرضة لأخطاء الترتيب.	الصنف StudentAccount.Builder
Singleton	يجب أن يوجد مستودع واحد وسجل تدقيق واحد؛ وإلا ضاعت الحسابات بين النسخ.	AccountRepository و AuditLogger
Prototype	مئات الطلبة يلتحقون بنفس التخصص والمستوى والمقررات؛ بناء كل حساب من الصفر تكرار لا داعي له.	StudentAccount.copy() و AccountTemplate

حقول الصنف StudentAccount

الحقل	النوع	إجباري؟	القاعدة / القيمة الافتراضية
id	String	نعم	غير فارغ، وفريد في المستودع
fullName	String	نعم	غير فارغ
email	String	نعم	يحتوي على الرمز @
phone	String	لا	null
level	String	لا	من L1, L2, L3, M1, M2 — الافتراضي L1
major	String	لا	التخصص
scholarship	boolean	لا	false
courses	List<String>	لا	قائمة فارغة

3. مخطط الأصناف (UML)



الشكل 1: مخطط الأصناف للأنماط الثلاثة وعلاقاتها (لون الصنف = النمط الذي يمثله)

قراءة المخطط

- **Builder:** الصنف Builder داخل StudentAccount ينشئه عبر build() (علاقة اعتماد «creates»)، ومُنشئ private StudentAccount فلا طريق آخر للإنشاء.
- **Singleton:** لكل من AccountRepository و AuditLogger مُنشئ private ودالة getInstance() وصنف داخلي Holder يحمل النسخة الوحيدة. المستودع يخزّن عدداً من الحسابات (تجميع 0..*) ويستعمل AuditLogger لتسجيل كل حفظ.
- **Prototype:** AccountTemplate يحتفظ بحساب نموذجي وينتج حسابات جديدة بنداء copy() ثم changeIdentity(). الصنف StudentAccount يجمع دورين: منتَج الـ Builder ونموذج الـ Prototype (ولهذا رأسه بلونين).

4. شرح الأنماط

Builder بناء الحساب

أين استعملته؟ في الصنف StudentAccount عبر صنفه الداخلي Builder (دور ConcreteBuilder)، وبه يُبنى كل حساب في Main.

ما المشكلة التي حلها؟ للحساب ثمانية حقول، ثلاثة إجبارية وخمسة اختيارية. مُنشئ واحد بثمانية وسائط غير مقروء، ويسهل فيه تبديل وسيطين من نفس النوع (مثل phone و major وكلاهما String) دون أن يلاحظ المترجم. في حلنا: الحقول الإجبارية الثلاثة في مُنشئ Builder فيستحيل نسيانها، والاختيارية بدوال متسلسلة بقيم افتراضية، والدالة build() تتحقق مرة واحدة (id غير فارغ، البريد فيه @، المستوى من القائمة) قبل ولادة الكائن، فلا يوجد حساب غير صالح.

ماذا لو غاب؟ مُنشئ بثمانية وسائط (أو عدة مُنشئات متراكبة)، وكود غير مقروء، وأخطاء ترتيب صامتة، وتحقق مبعثر قد يسمح بحساب ببريد خاطئ.

```
StudentAccount a1 = new StudentAccount.Builder("20240001", "Yacine Bensalem",
"y.bensalem@univ.dz")
        .level("L2").major("Computer Science").scholarship(true)
        .addCourse("Algorithms").addCourse("Databases").build();
```

Singleton المستودع وسجل التدقيق

أين استعملته؟ في صنفين: AccountRepository (المستودع المركزي) و AuditLogger (سجل التدقيق)، وكلاهما دور Singleton.

ما المشكلة التي حلها؟ لو أنشأ كل جزء من البرنامج مستودعه الخاص لضاعت الحسابات بين النسخ ولم يمكن ضمان أن الـ id فريد، ولتبعثر سجل العمليات. الحل: مُنشئ private (لا أحد يكتب new)، ودالة getInstance() هي نقطة الوصول الوحيدة، وصنف داخلي Holder يحمل النسخة فتُنشأ عند أول استعمال وبأمان مع تعدد الخيوط دون synchronized على الدالة. وتتعاون النسختان: كل save() ناجحة تكتب سطراً في السجل، والـ id المكرر يُرفض باستثناء.

ماذا لو غاب؟ عدة مستودعات: حساب محفوظ في نسخة لا تراه نسخة أخرى، وإمكانية تكرار الـ id، وسجل تدقيق مجزأ لا يعبّر عن كل العمليات.

```
private AuditLogger() { }   // nobody can call "new AuditLogger()"
public static AuditLogger getInstance() { return Holder.INSTANCE; }
```

Prototype قوالب الدفعات

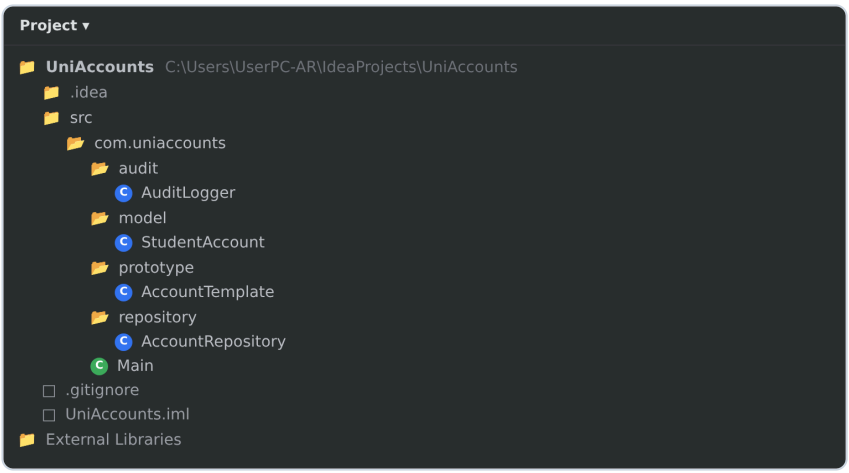
أين استعملته؟ في الدالة StudentAccount.copy() (دور ConcretePrototype) وفي الصنف AccountTemplate الذي يحتفظ بالنموذج وينتج النسخ، مع الدالة changeIdentity() لتغيير الهوية.

ما المشكلة التي حلها؟ في بداية كل سنة يلتحق مئات الطلبة بنفس التخصص والمستوى ولهم نفس المقررات. بدل بناء كل حساب من الصفر نُحضّر قالباً لكل دفعة (مثلاً L2 إعلام آلي)، وننسخه ونغيّر الـ id والاسم والبريد فقط. **نقطة مهمة:** النسخ يجب أن يكون **عميقاً** لقائمة المقررات؛ فالدالة clone() وحدها تنسخ المرجع فتتشارك الحسابات نفس القائمة، ولهذا تنشئ copy() قائمة جديدة.

ماذا لو غاب؟ نكرر بناء الحساب بنفس المقررات مئات المرات عبر Builder، ويزيد احتمال اختلاف حسابات الدفعة الواحدة. ولو كان النسخ سطحياً لتداخلت قوائم المقررات: تعديل حساب يغيّر حسابات أخرى.

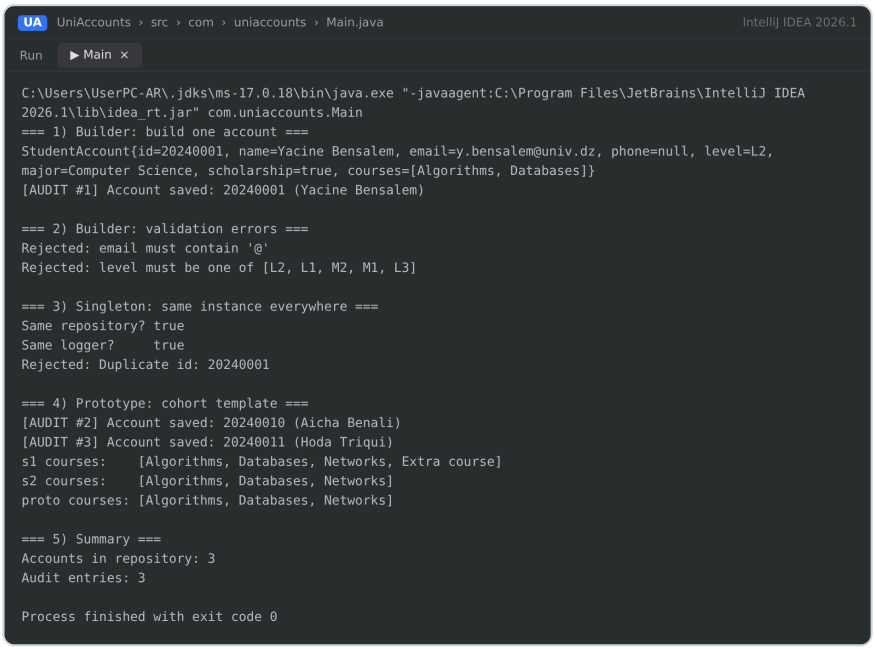
```
public StudentAccount copy() {
    StudentAccount clone = (StudentAccount) super.clone();
    clone.courses = new ArrayList<>(this.courses);   // deep copy of the list
    return clone;
}
StudentAccount s1 = l2cs.createFor("20240010", "Aicha Benali", "aicha@univ.dz");
```

5. هيكل المشروع



الشكل 2: شجرة المشروع في IntelliJ IDEA — الحزم الأربع وصنف Main

6. نتيجة التشغيل



الشكل 3: ناتج تشغيل Main (انتهى البرنامج بالرمز exit code 0)

القسم في الناتج	ما يثبته
1) Builder	بناء حساب كامل بدوال متسلسلة وحفظه؛ ظهر السطر [AUDIT #1] في السجل.
2) أخطاء التحقق	رُفض حساب ببريد بلا @ وحساب بمستوى غير مسموح (L9) قبل إنشائهما.
3) Singleton	المستودع والسجل نفس الكائن (true و true)، ورُفض الـ id المكرر 20240001.
4) Prototype	أُنشئ حسابان من قالب L2؛ مقرر إضافي لـ s1 لم يظهر في s2 ولا في القالب، وهو دليل النسخ العميق.
5) الملخص	3 حسابات في المستودع و3 إدخالات في سجل التدقيق، فالعمليتان متطابقتان.

7. طريقة التشغيل والخاتمة

التشغيل: افتح المشروع في IntelliJ IDEA (JDK 17 فما فوق)، ثم شغّل الصنف com.uniaccounts.Main. أو من سطر الأوامر:

```
javac -d out $(find src -name "*.java")
java -cp out com.uniaccounts.Main
```

الخاتمة: وفّرت الأنماط الثلاثة نواة سليمة قابلة للتوسعة في العمل الثاني: Builder ضمن حساباً صالحاً ومقروءاً، وSingleton وحّد المستودع والسجل، وPrototype سرّع إنشاء حسابات الدفعة الواحدة. وسنعيد استعمال هذه النواة في العمل الثاني دون إعادة كتابتها.

المصادر:

1) درس نماذج التصميم للمقياس.

2) E. Gamma, R. Helm, R. Johnson, J. Vlissides — *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994.

3) J. Bloch — *Effective Java* (Item: Builder); Oracle Java documentation.

4) أداة Claude (Anthropic) للمساعدة في شرح الأنماط وتوليد المسودة الأولى للكود؛ وقد راجعتُ الكود وجرّبته وأفهمه.